

## ✓ Import Libraries

We import essential libraries for data manipulation, text processing, machine learning models, deep learning layers, evaluation metrics, and visualization.

```
# ===== Import Libraries =====

# Basic data manipulation libraries
import pandas as pd          # for dataframes and data manipulation
import numpy as np           # for numerical operations

# Text preprocessing libraries
import re                    # for regular expressions
import string                # for string operations
import nltk                  # for natural language processing tasks
from nltk.corpus import stopwords  # for stopwords removal
from nltk.stem import WordNetLemmatizer  # for lemmatization

# Machine learning models
from sklearn.linear_model import LogisticRegression  # logistic regression
from sklearn.ensemble import RandomForestClassifier  # random forest
from sklearn.svm import SVC  # support vector machine
from sklearn.naive_bayes import MultinomialNB  # naive bayes
from xgboost import XGBClassifier  # XGBoost
from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier  # boosting models

# Deep learning (TensorFlow / Keras) libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential  # sequential model
from tensorflow.keras.layers import Embedding, LSTM, GRU, Dense, Dropout  # neural network layers
from tensorflow.keras.preprocessing.text import Tokenizer  # tokenizer for text
from tensorflow.keras.preprocessing.sequence import pad_sequences  # sequence padding

# Feature extraction methods
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer  # bag of words, TF-IDF

# Evaluation metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

# Model selection and hyperparameter tuning
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV

# Visualization libraries
import matplotlib.pyplot as plt  # for plotting
import seaborn as sns            # for enhanced plots

# Suppress warnings to keep the output clean
import warnings
warnings.filterwarnings('ignore')

nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
↳ [nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
True
```

## ✓ Load Dataset

We load the Amazon product review dataset containing review texts and their sentiment labels (positive/negative).

```
df = pd.read_csv('/content/amazon (1).csv')
```

```
df.head(20)
```



|    | reviewText                                        | Positive |  |
|----|---------------------------------------------------|----------|-----------------------------------------------------------------------------------|
| 0  | This is a one of the best apps acording to a b... | 1        |  |
| 1  | This is a pretty good version of the game for ... | 1        |                                                                                   |
| 2  | this is a really cool game. there are a bunch ... | 1        |                                                                                   |
| 3  | This is a silly game and can be frustrating, b... | 1        |                                                                                   |
| 4  | This is a terrific game on any pad. Hrs of fun... | 1        |                                                                                   |
| 5  | This is a very entertaining game! You don't h...  | 1        |                                                                                   |
| 6  | this is awesome and you don't need wi ti to pl... | 1        |                                                                                   |
| 7  | this is awesome I bet no one even reads the re... | 1        |                                                                                   |
| 8  | This is basicly the free version but with ads.... | 1        |                                                                                   |
| 9  | this is by far the best free app that is avail... | 1        |                                                                                   |
| 10 | This is definitely a great game. I have to ge...  | 1        |                                                                                   |
| 11 | this. is fun an time consuming. works great on... | 1        |                                                                                   |
| 12 | This is good if you like physics games, free g... | 1        |                                                                                   |
| 13 | This is great! This is my tried & true game to... | 1        |                                                                                   |
| 14 | this is one of the best apps in the world alon... | 1        |                                                                                   |
| 15 | This is super fun though a little frustrating ... | 1        |                                                                                   |
| 16 | this is the ad free version of angry birds. th... | 1        |                                                                                   |
| 17 | This is the best app ever. It makes it way eas... | 1        |                                                                                   |
| 18 | this is the best app I ever downloaded for mos... | 1        |                                                                                   |
| 19 | THIS IS THE BESTEST GAME EVER OMG. IM SO EXCIT... | 1        |                                                                                   |

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

```
df.shape
```



```
(20000, 2)
```

```
df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   reviewText  20000 non-null  object
1   Positive    20000 non-null  int64
dtypes: int64(1), object(1)
memory usage: 312.6+ KB
```

```
df.isnull().sum()
```



```
0
reviewText  0
Positive    0
```

```
df.tail()
```

|       | reviewText                                        | Positive |
|-------|---------------------------------------------------|----------|
| 19995 | this app is fricken stupid.it froze on the kin... | 0        |
| 19996 | Please add me!!!! I need neighbors! Ginger101...  | 1        |
| 19997 | love it! this game. is awesome. wish it had m...  | 1        |
| 19998 | I love love love this app on my side of fashio... | 1        |
| 19999 | This name is a rin off Here is a list of thin     | 0        |

## ✓ Check Class Distribution

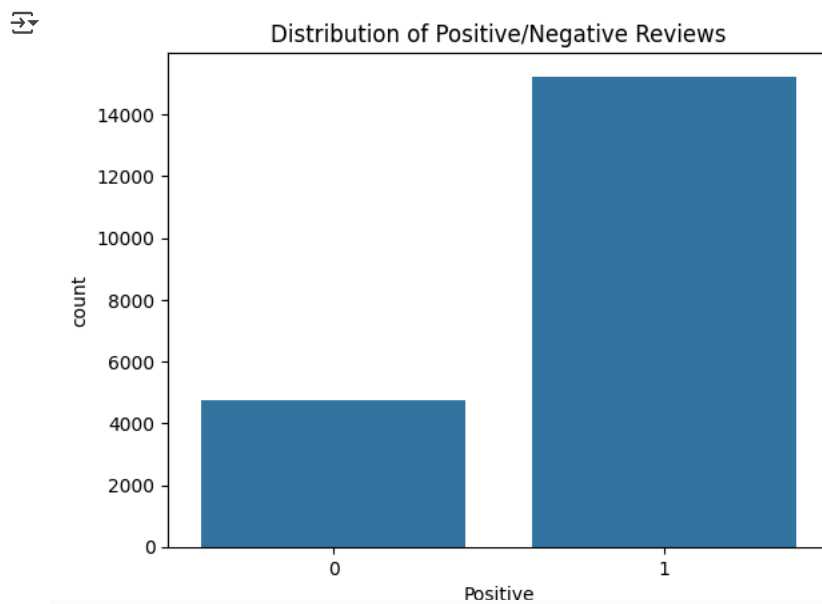
We plot the distribution of positive and negative reviews to check for class imbalance.

```
df['Positive'].value_counts()
```

|          | count |
|----------|-------|
| Positive |       |
| 1        | 15233 |
| 0        | 4767  |

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='Positive', data=df)
plt.title('Distribution of Positive/Negative Reviews')
plt.show()
```



```
df['review_length'] = df['reviewText'].apply(len)
df['review_length'].describe()
```

|       | review_length |
|-------|---------------|
| count | 20000.000000  |
| mean  | 175.793850    |
| std   | 58.966619     |
| min   | 3.000000      |
| 25%   | 123.000000    |
| 50%   | 164.000000    |
| 75%   | 254.000000    |
| max   | 254.000000    |

```
df.head()
```

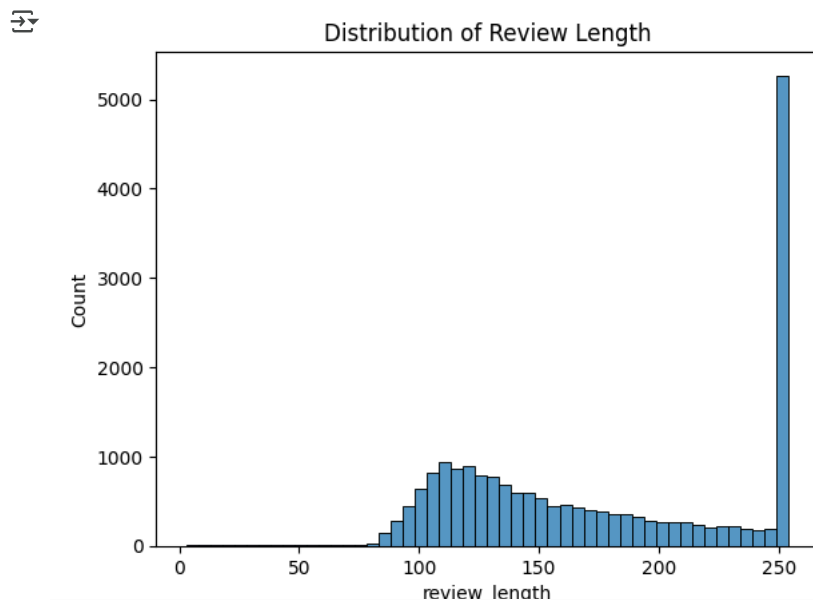
|   | reviewText                                         | Positive | review_length |
|---|----------------------------------------------------|----------|---------------|
| 0 | This is a one of the best apps according to a b... | 1        | 129           |
| 1 | This is a pretty good version of the game for ...  | 1        | 129           |
| 2 | this is a really cool game. there are a bunch ...  | 1        | 96            |
| 3 | This is a silly game and can be frustrating, b...  | 1        | 105           |
| 4 | This is a terrific game on any pad. Hrs of fun     | 1        | 117           |

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

## ✓ Add Review Length and Word Count Features

We compute additional features — the length of each review and the number of words — to explore their distributions and potential correlation with sentiment.

```
sns.histplot(df['review_length'], bins=50)
plt.title('Distribution of Review Length')
plt.show()
```

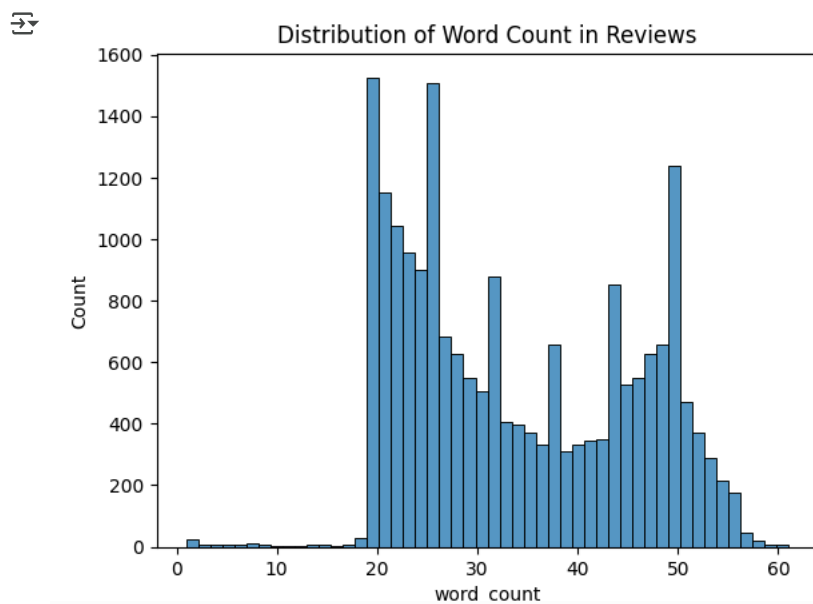


```
df['word_count'] = df['reviewText'].apply(lambda x: len(str(x).split()))
df['word_count'].describe()
```

```
word_count
```

|       | word_count   |
|-------|--------------|
| count | 20000.000000 |
| mean  | 33.941450    |
| std   | 11.217532    |
| min   | 1.000000     |
| 25%   | 24.000000    |
| 50%   | 31.000000    |
| 75%   | 45.000000    |
| max   | 61.000000    |

```
sns.histplot(df['word_count'], bins=50)
plt.title('Distribution of Word Count in Reviews')
plt.show()
```



## Word Frequency Analysis

We analyze the most common words across all reviews after removing stopwords to understand key patterns in the text data.

```
# ===== Word Frequency Analysis =====

# Import Counter to count word frequencies
from collections import Counter

# Import nltk and stopwords
import nltk
from nltk.corpus import stopwords

# Download the NLTK stopwords list
nltk.download('stopwords')

# Create a set of English stopwords
stop_words = set(stopwords.words('english'))

# Combine all review texts into a single string, convert to lowercase, and split into words
all_words = ' '.join(df['reviewText']).lower().split()

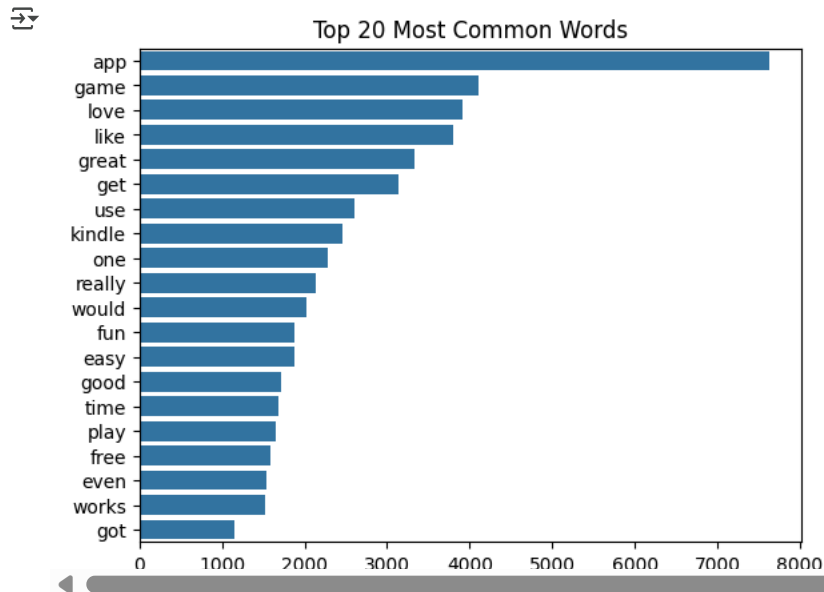
# Filter words: keep only alphabetic words and remove stopwords
filtered_words = [word for word in all_words if word.isalpha() and word not in stop_words]

# Get the 20 most common words and their frequencies
common_words = Counter(filtered_words).most_common(20)
```

```
common_words = Counter(filtered_words).most_common(20)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
words, counts = zip(*common_words)
sns.barplot(x=list(counts), y=list(words))
plt.title('Top 20 Most Common Words')
plt.show()
```



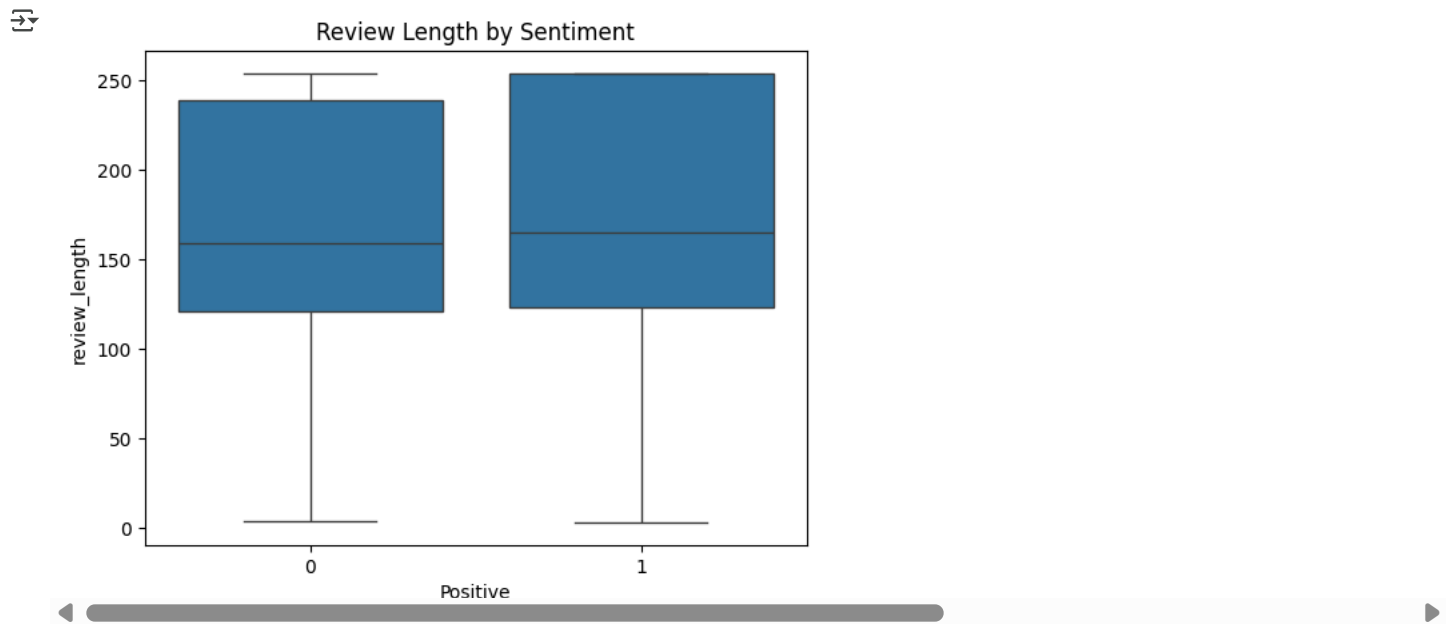
## ✓ Analyze Review Length by Sentiment

We compare average review length and word count between positive and negative reviews to explore linguistic differences.

```
df.groupby('Positive')['review_length'].mean()
df.groupby('Positive')['word_count'].mean()
```

|          | word_count |
|----------|------------|
| Positive |            |
| 0        | 33.254248  |
| 1        | 34.156502  |

```
sns.boxplot(x='Positive', y='review_length', data=df)
plt.title('Review Length by Sentiment')
plt.show()
```



```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df['clean_text'] = df['reviewText'].str.lower()
```

```
df.head()
```

|   | reviewText                                         | Positive | review_length | word_count | clean_text                                         |
|---|----------------------------------------------------|----------|---------------|------------|----------------------------------------------------|
| 0 | This is a one of the best apps according to a b... | 1        | 129           | 28         | this is a one of the best apps according to a b... |
| 1 | This is a pretty good version of the game for ...  | 1        | 129           | 27         | this is a pretty good version of the game for ...  |
| 2 | this is a really cool game. there are a bunch ...  | 1        | 96            | 20         | this is a really cool game. there are a bunch ...  |
| 3 | This is a silly game and can be frustrating, b...  | 1        | 105           | 21         | this is a silly game and can be frustrating, b...  |
| 4 | This is a terrific game on any pad. Hrs of fun     | 1        | 117           | 22         | this is a terrific game on any pad. hrs of fun     |

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

## ▼ Data Cleaning

We clean the text by lowercasing, removing HTML tags, punctuation, numbers, and stopwords, preparing it for modeling.

```
import re
import string
from nltk.corpus import stopwords
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower() # lowercase
    text = re.sub(r'<.*?>', '', text) # remove HTML tags
    text = re.sub(r'[\s\w]', '', text) # remove punctuation
    text = re.sub(r'\d+', '', text) # remove numbers
    text = re.sub(r'\s+', ' ', text) # remove extra spaces
    text = ' '.join([word for word in text.split() if word not in stop_words]) # remove stopwords
    return text

df['clean_review'] = df['reviewText'].apply(clean_text)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

Encode Sentiment Labels

We convert the `Positive` label from categorical to numeric format using `LabelEncoder`.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Positive'] = le.fit_transform(df['Positive'])
```

```
df.head()
```

|   | reviewText                                           | Positive | review_length | word_count | clean_text                                           | clean_review                                         |
|---|------------------------------------------------------|----------|---------------|------------|------------------------------------------------------|------------------------------------------------------|
| 0 | This is a one of the best apps<br>acording to a b... | 1        | 129           | 28         | this is a one of the best apps<br>acording to a b... | one best apps acording bunch people<br>agree bomb... |
| 1 | This is a pretty good version of<br>the game for ... | 1        | 129           | 27         | this is a pretty good version of<br>the game for ... | pretty good version game free lots<br>different l... |
| 2 | this is a really cool game. there<br>are a bunch ... | 1        | 96            | 20         | this is a really cool game. there<br>are a bunch ... | really cool game bunch levels find<br>golden eggs... |

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

Apply Lemmatization

We lemmatize words to reduce them to their base form, improving the quality of the features.

```
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()
def lemmatize_text(text):
    return ' '.join([lemmatizer.lemmatize(word) for word in text.split()])

df['clean_review'] = df['clean_review'].apply(lemmatize_text)
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

```
df[['reviewText', 'clean_review']].sample(5)
```

|       | reviewText                                        | clean_review                                      |
|-------|---------------------------------------------------|---------------------------------------------------|
| 15774 | Because I have added so many apps in the Appst... | added many apps appstore quotapp dayquot neede... |
| 9032  | Task managers are pathetic and are based on th... | task manager pathetic based assumption phone d... |
| 13024 | This app is super sweet - it's a star map that... | app super sweet star map know direction youre ... |
| 13349 | I really LOVED this app. Very fun and interact... | really loved app fun interactive thinking gett... |
| 4100  | The ann is fine But it is being down voted by     | ann fine voted moron never used amazon filter     |

```
df['Positive'].value_counts()
```

|          | count |
|----------|-------|
| Positive |       |
| 1        | 15233 |
| 0        | 4767  |

df.head()

|   | reviewText                                           | Positive | review_length | word_count | clean_text                                           | clean_review                                         |
|---|------------------------------------------------------|----------|---------------|------------|------------------------------------------------------|------------------------------------------------------|
| 0 | This is a one of the best apps<br>acording to a b... | 1        | 129           | 28         | this is a one of the best apps<br>acording to a b... | one best apps acording bunch people<br>agree bomb... |
| 1 | This is a pretty good version of<br>the game for ... | 1        | 129           | 27         | this is a pretty good version of<br>the game for ... | pretty good version game free lot<br>different le... |
| 2 | this is a really cool game. there<br>are a bunch ... | 1        | 96            | 20         | this is a really cool game. there<br>are a bunch ... | really cool game bunch level find<br>golden egg s... |

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

▼ Apply Lemmatization

We lemmatize words to reduce them to their base form, improving the quality of the features.

```
# ===== TF-IDF Vectorization =====

# Import TfidfVectorizer to convert text data into numerical features
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize the TF-IDF vectorizer with a maximum of 5000 features (most important words)
tfidf = TfidfVectorizer(max_features=5000)

# Transform the 'clean_review' column into a TF-IDF feature matrix (X)
X = tfidf.fit_transform(df['clean_review'])

# Set the target variable (y) as the 'Positive' column
y = df['Positive']
```

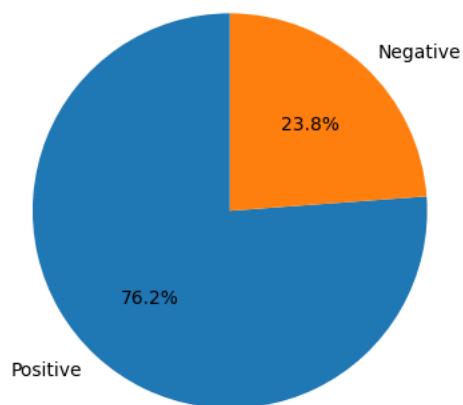
▼ Plot Class Distribution

We visualize the balance of positive and negative classes before and after applying SMOTE oversampling.

```
y.value_counts().plot(kind='pie', autopct='%1.1f%%', startangle=90, labels=['Positive', 'Negative'])
plt.title('Class Distribution Before SMOTE')
plt.ylabel('') # Remove y-axis label for clean look
plt.show()
```



Class Distribution Before SMOTE



## ▼ Train-Test Split

We split the dataset into training and testing sets to prepare for modeling and evaluation.

### Apply SMOTE for Class Balancing

We use SMOTE to oversample the minority class in the training data, improving the model's ability to handle imbalance.

```
# ===== Train-Test Split =====

# Split the feature matrix X and target y into training and testing sets
# Use stratified splitting to maintain the class distribution in both sets
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# ===== Apply SMOTE =====

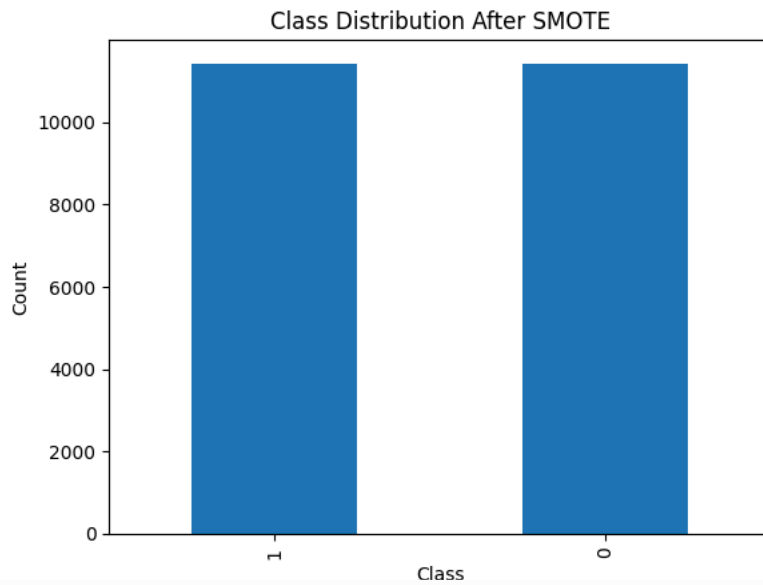
# Import SMOTE from imblearn for oversampling the minority class
from imblearn.over_sampling import SMOTE

# Initialize the SMOTE object with a fixed random state for reproducibility
smote = SMOTE(random_state=42)

# Apply SMOTE only on the training data to balance the class distribution
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

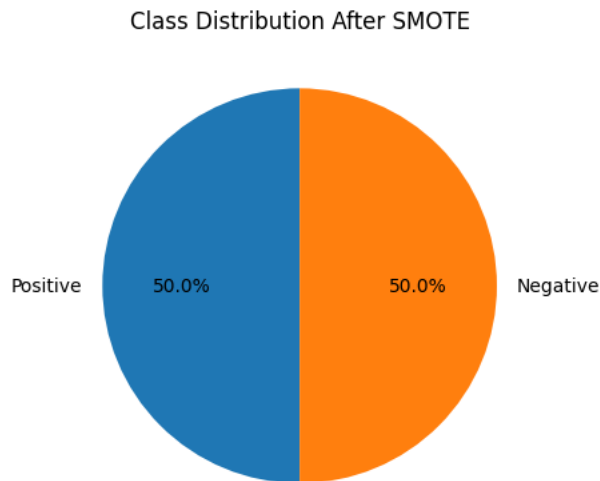
import pandas as pd

pd.Series(y_train_resampled).value_counts().plot(kind='bar')
plt.title('Class Distribution After SMOTE')
plt.xlabel('Class')
plt.ylabel('Count')
plt.show()
```



```
import pandas as pd

pd.Series(y_train_resampled).value_counts().plot(
    kind='pie', autopct='%1.1f%%', startangle=90, labels=['Positive', 'Negative']
)
plt.title('Class Distribution After SMOTE')
plt.ylabel('')
plt.show()
```



## ✓ Train and Evaluate Machine Learning Models

We train multiple ML models (Logistic Regression, Random Forest, SVM, Naive Bayes, XGBoost), evaluate them with accuracy and classification reports, and visualize confusion matrices and ROC curves.

```
# ===== 1. DATA SPLITTING =====
from sklearn.metrics import roc_curve, auc

# Split the dataset into train and test sets for deep learning models (raw text)
X_train_text, X_test_text, y_train, y_test = train_test_split(
    df['clean_review'], df['Positive'], test_size=0.2, random_state=42, stratify=df['Positive']
)

# Create TF-IDF features for machine learning models
```

```

tfidf = TfidfVectorizer(max_features=5000)
X_tfidf = tfidf.fit_transform(df['clean_review'])

# Split the TF-IDF matrix into train and test sets for ML
X_train_ml, X_test_ml, _, _ = train_test_split(
    X_tfidf, df['Positive'], test_size=0.2, random_state=42, stratify=df['Positive']
)

# ===== 2. MACHINE LEARNING MODELS =====

# Define a dictionary of ML models to train and evaluate
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Random Forest': RandomForestClassifier(),
    'SVM (Linear Kernel)': SVC(kernel='linear', probability=True),
    'Naive Bayes': MultinomialNB(),
    'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='logloss')
}

# Loop over each model, train it, evaluate it, and plot metrics
for name, model in models.items():
    # Train the model
    model.fit(X_train_ml, y_train)

    # Make predictions
    y_pred = model.predict(X_test_ml)

    # Get predicted probabilities for ROC curve
    y_proba = model.predict_proba(X_test_ml)[:,1] if hasattr(model, 'predict_proba') else model.decision_function(X_test_ml)
    y_proba = (y_proba - y_proba.min()) / (y_proba.max() - y_proba.min()) # Scale to [0,1] if needed

    # Print evaluation metrics
    print(f"\n==== Model: {name} =====")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))

    # Plot confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f'{name} - Confusion Matrix')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()

    # Plot ROC curve
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(6,5))
    plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
    plt.plot([0,1],[0,1], 'k--') # Diagonal reference line
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'{name} - ROC Curve')
    plt.legend()
    plt.show()

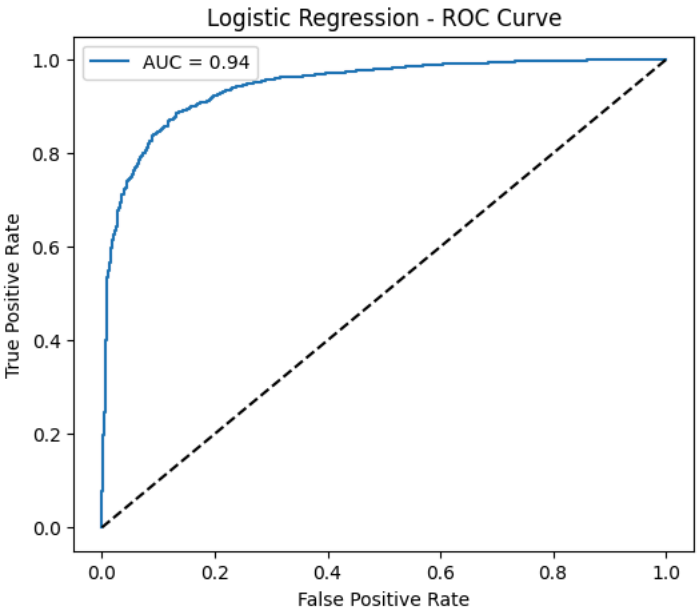
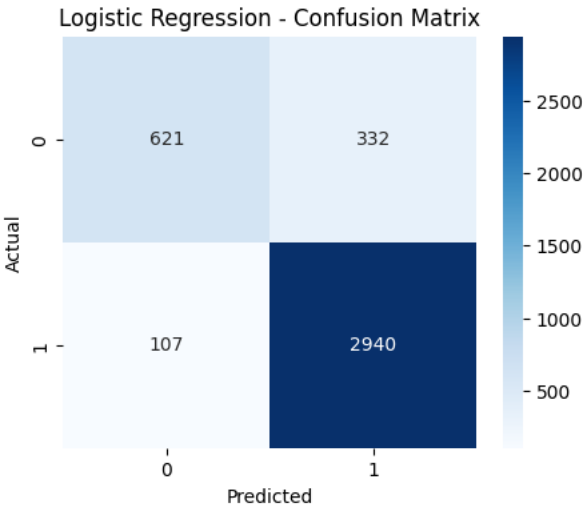
```



```
==== Model: Logistic Regression ====
Accuracy: 0.89025
Classification Report:
              precision    recall  f1-score   support

    0           0.85       0.65       0.74         953
    1           0.90       0.96       0.93        3047

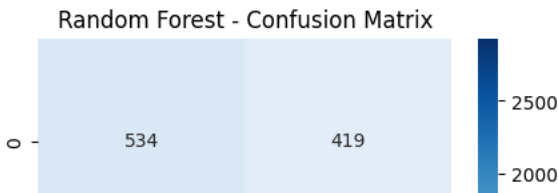
 accuracy          0.89          0.89          0.88         4000
 macro avg         0.88         0.81         0.83         4000
 weighted avg      0.89         0.89         0.88         4000
```

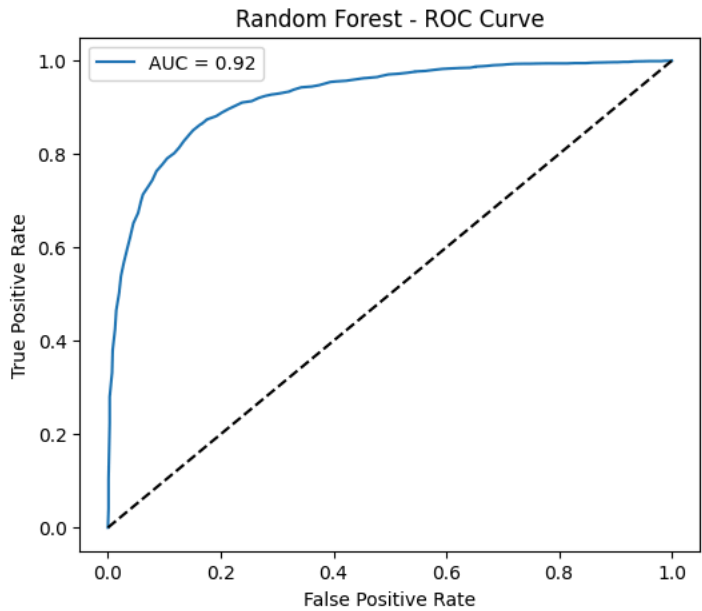
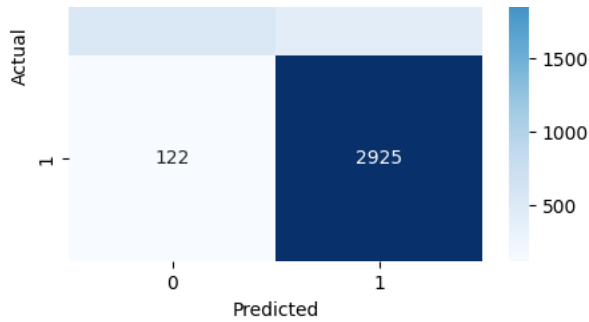


```
==== Model: Random Forest ====
Accuracy: 0.86475
Classification Report:
              precision    recall  f1-score   support

    0           0.81       0.56       0.66         953
    1           0.87       0.96       0.92        3047

 accuracy          0.86          0.86          0.86         4000
 macro avg         0.84         0.76         0.79         4000
 weighted avg      0.86         0.86         0.86         4000
```



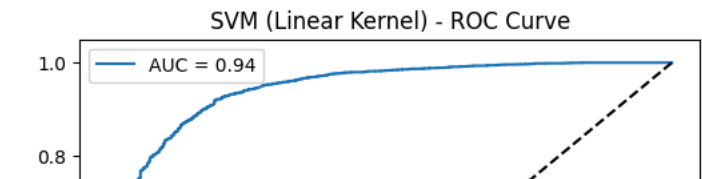
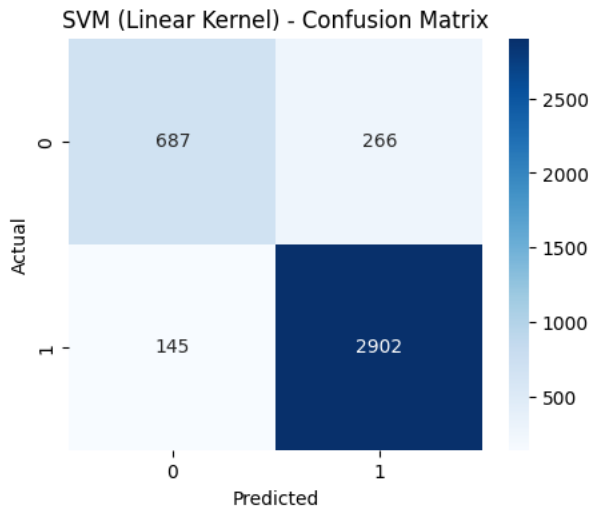


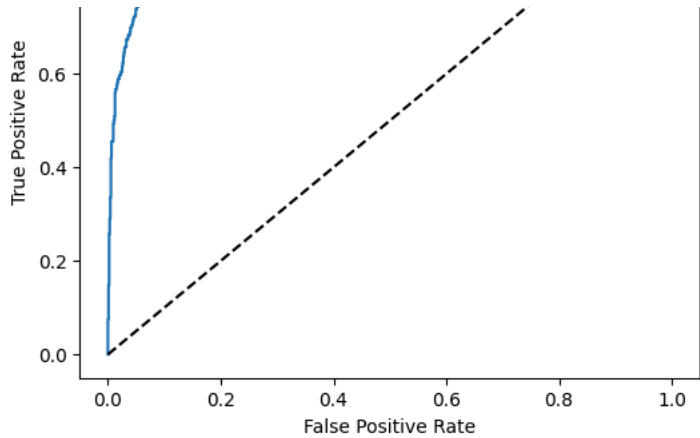
==== Model: SVM (Linear Kernel) =====

Accuracy: 0.89725

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.83      | 0.72   | 0.77     | 953     |
| 1            | 0.92      | 0.95   | 0.93     | 3047    |
| accuracy     |           |        | 0.90     | 4000    |
| macro avg    | 0.87      | 0.84   | 0.85     | 4000    |
| weighted avg | 0.89      | 0.90   | 0.89     | 4000    |



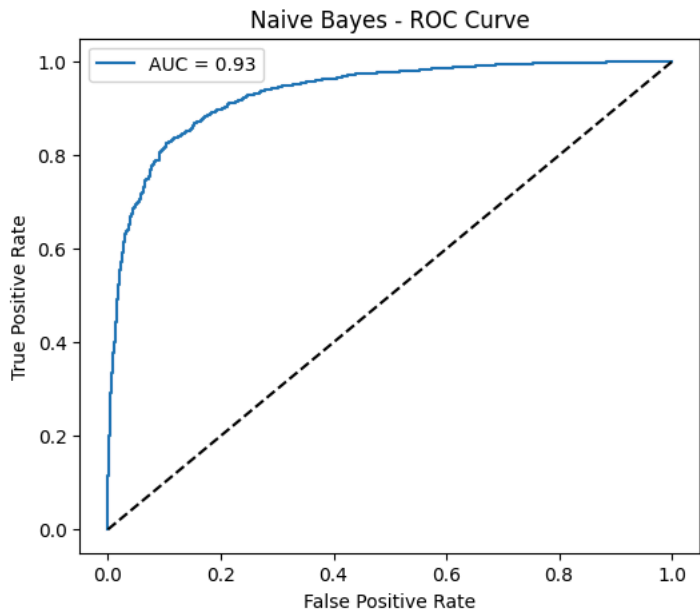
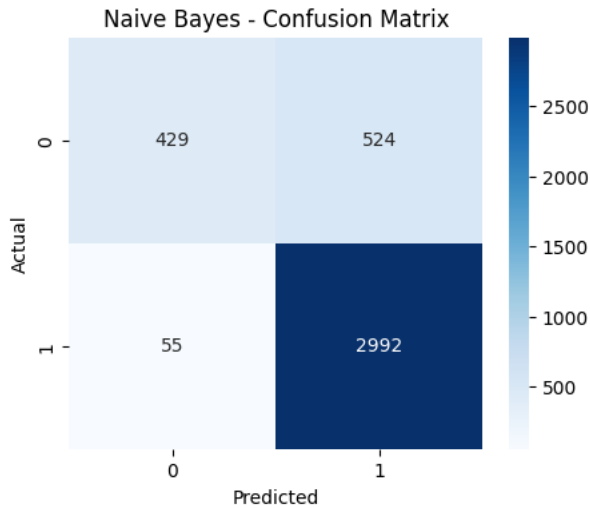


==== Model: Naive Bayes ====

Accuracy: 0.85525

Classification Report:

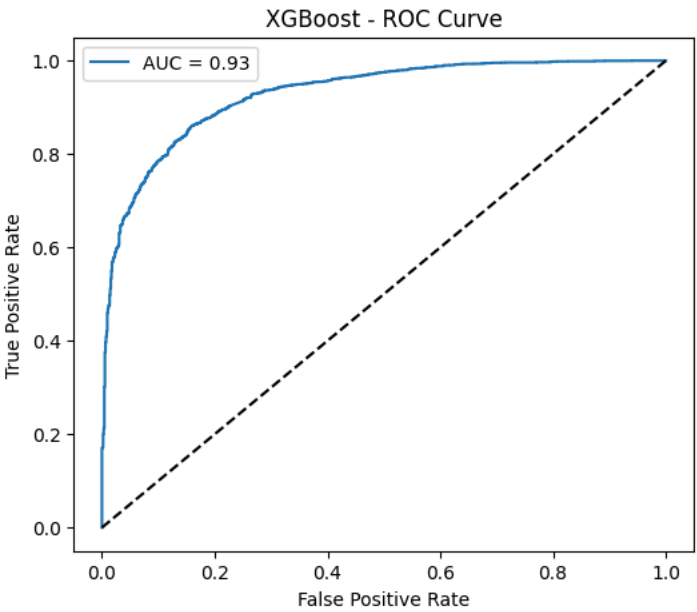
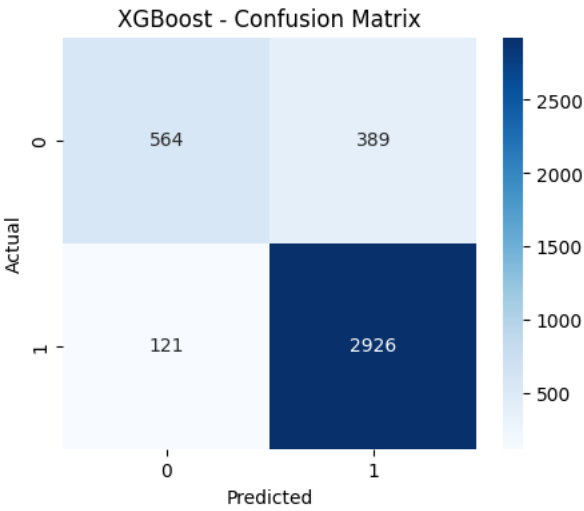
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.89      | 0.45   | 0.60     | 953     |
| 1            | 0.85      | 0.98   | 0.91     | 3047    |
| accuracy     |           |        | 0.86     | 4000    |
| macro avg    | 0.87      | 0.72   | 0.75     | 4000    |
| weighted avg | 0.86      | 0.86   | 0.84     | 4000    |



==== Model: XGBoost ====

Accuracy: 0.8725

| Classification Report: |           |        |          |         |  |
|------------------------|-----------|--------|----------|---------|--|
|                        | precision | recall | f1-score | support |  |
| 0                      | 0.82      | 0.59   | 0.69     | 953     |  |
| 1                      | 0.88      | 0.96   | 0.92     | 3047    |  |
| accuracy               |           |        | 0.87     | 4000    |  |
| macro avg              | 0.85      | 0.78   | 0.80     | 4000    |  |
| weighted avg           | 0.87      | 0.87   | 0.86     | 4000    |  |



## ▼ Prepare Data for Deep Learning Models

We tokenize and pad the review texts to prepare inputs for LSTM and GRU models.

```
# Import necessary libraries
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, GRU, Dense, Dropout, Bidirectional
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt

# ===== Data Preparation =====

# Initialize tokenizer with a vocabulary size of 10,000 and OOV token for unknown words
tokenizer = Tokenizer(num_words=10000, oov_token='<OOV>')

# Fit tokenizer on training text
tokenizer.fit_on_texts(X_train_text)

# Convert text to integer sequences
X_train_seq = tokenizer.texts_to_sequences(X_train_text)
X_test_seq = tokenizer.texts_to_sequences(X_test_text)

# Pad sequences to have the same length
max_len = 100
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post')

# Set up early stopping to prevent overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# ===== Model Training and Evaluation Function =====

def train_dl_model(model_name, RNN_layer):
    # Define sequential model architecture
    model = Sequential([
        Embedding(input_dim=10000, output_dim=64, input_length=max_len),
        Bidirectional(RNN_layer), # Use Bidirectional wrapper for LSTM or GRU
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])

    # Compile the model
    model.compile(loss='binary_crossentropy', optimizer=Adam(0.001), metrics=['accuracy'])

    # Train the model
    print(f"\n==== Training {model_name} =====")
    history = model.fit(
        X_train_pad, y_train,
        epochs=15, batch_size=32,
        validation_data=(X_test_pad, y_test),
        callbacks=[early_stop],
        verbose=2
    )

    # Make predictions on the test set
    y_pred_prob = model.predict(X_test_pad)
    y_pred = (y_pred_prob > 0.5).astype('int32').flatten()

    # Print classification metrics
    print(f"\n==== {model_name} Evaluation =====")
    print(classification_report(y_test, y_pred))

    # Calculate and print confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    print("Confusion Matrix:\n", cm)

    # Plot accuracy over epochs
    plt.plot(history.history['accuracy'], label=f'{model_name} Train')
    plt.plot(history.history['val_accuracy'], label=f'{model_name} Val')
    plt.title(f'{model_name} Accuracy')
    plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# Plot confusion matrix as heatmap
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[0,1], yticklabels=[0,1])
plt.title(f'{model_name} - Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# ===== Run Bidirectional LSTM =====

from tensorflow.keras.layers import LSTM as LSTM_layer
train_dl_model('Bidirectional LSTM', LSTM_layer(64))
```